Алгоритмы сортировки

Алгоритмы сортировки

Сортировка — это процесс упорядочивания элементов в заданном порядке (обычно по возрастанию или убыванию).

Ключевые понятия:

- Ключ сортировки — поле, по которому выполняется упорядочивание
- Устойчивость (стабильность) — сохранение относительного порядка элементов с равными ключами

Зачем нужна сортировка?

- Ускорение поиска (бинарный поиск работает только на отсортированных данных)
- Предсказуемое поведение систем
- Основа для многих других алгоритмов (поиск, обработка графов, сжатие данных)

Ключевые понятия оценки алгоритмов

Понятие	Описание	Примеры
Временная сложность	Как быстро растёт время выполнения при увеличении размера данных	$O(n)$, $O(n^2)$, $O(n \log n)$
Пространственная сложность	Сколько дополнительной памяти требует алгоритм	$O(1)$ — не требует доп. памяти, $O(n)$ — требует копию массива
Стабильность	Сохраняется ли порядок равных элементов	Стабильные: пузырьковая, вставками, слиянием. Нестабильные: выбором, быстрая

Нотация Big O - математическое обозначение, описывающее поведение алгоритма (скорость работы или объем памяти) по мере роста объема входных данных (n).

Классификация алгоритмов сортировки

По месту выполнения:

- Внутренние — данные помещаются в оперативной памяти (большинство классических алгоритмов)
- Внешние — данные на внешних носителях, требуют специальных подходов

По принципу работы:

- Обменные (пузырьковая, быстрая)
- Выбора (выбором, пирамидальная)
- Вставками (вставками)
- Слиянием

По устойчивости:

- Стабильные: пузырьковая, вставками, слиянием
- Нестабильные: выбором, быстрая, пирамидальная

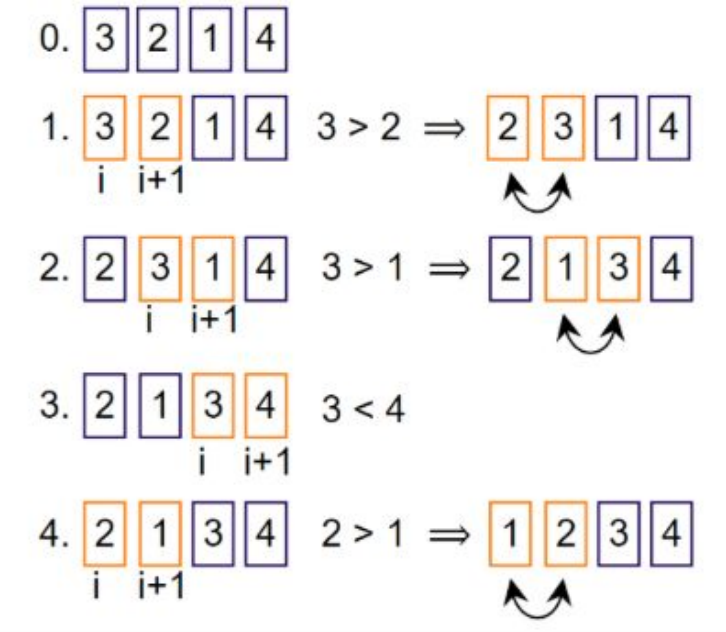
Пузырьковая сортировка (Bubble Sort)

Принцип: Попарное сравнение соседних элементов и их обмен, если они стоят в неправильном порядке. Более крупные элементы постепенно "всплывают" в конец списка.

Сначала сравниваются первые два элемента списка. Если первый элемент больше, они меняются местами. Если они уже в нужном порядке, оставляем их как есть. Затем переходим к следующей паре элементов, сравниваем их значения и меняем местами при необходимости. Этот процесс продолжается до последней пары элементов в списке.

При достижении конца списка процесс повторяется заново для каждого элемента. Это крайне неэффективно, если в массиве нужно сделать, например, только один обмен. Алгоритм повторяется n^2 раз, даже если список уже отсортирован.

Для оптимизации алгоритма нужно знать, когда его остановить, то есть когда список отсортирован.



Пузырьковая сортировка (Bubble Sort)

```
Исходный массив: [5, 3, 8, 1]
Шаг 1: 5>3 → [3, 5, 8, 1]
Шаг 2: 5<8 → [3, 5, 8, 1] (без обмена)
Шаг 3: 8>1 → [3, 5, 1, 8]
Шаг 4: 3<5 → [3, 5, 1, 8] (без обмена)
Шаг 5: 5>1 → [3, 1, 5, 8]
Шаг 6: 3>1 → [1, 3, 5, 8] — готово!
```

Пузырьковая сортировка (Bubble Sort)

Характеристики:

- Сложность: $O(n^2)$ — худший и средний, $O(n)$ — лучший (уже отсортированный массив)
- Память: $O(1)$ — дополнительная память не требуется
- Стабильность: Да

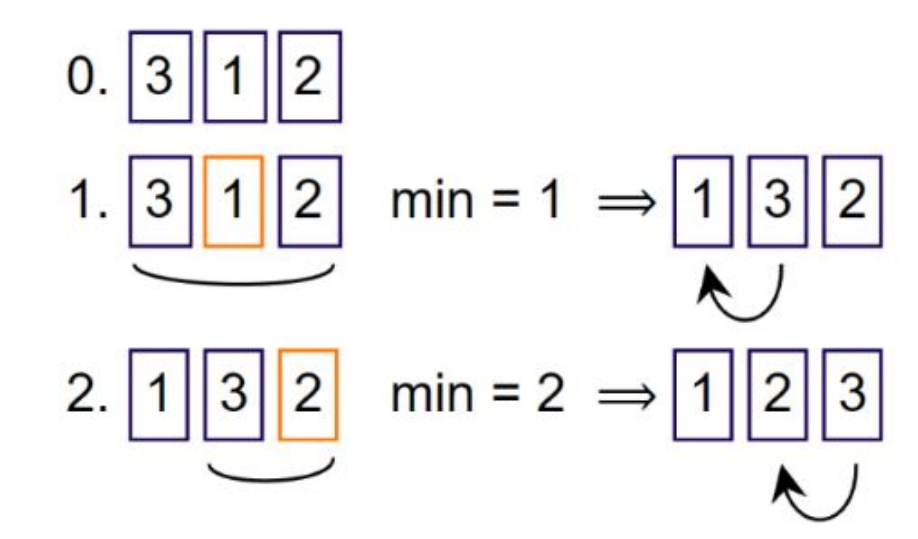
Сортировка выбором (Selection Sort)

Сортировка выбором — также простой алгоритм, но более эффективный по сравнению с пузырьковой сортировкой. В большинстве случаев сортировка выбором будет более удачным выбором из двух.

Этот алгоритм сегментирует список на две части: отсортированную и неотсортированную. Наименьший элемент удаляется из второго списка и добавляется в первый.

На практике не нужно создавать новый список для отсортированных элементов. В качестве него используется крайняя левая часть списка. Находится наименьший элемент и меняется с первым местами.

Теперь, когда нам известно, что первый элемент списка отсортирован, находим наименьший элемент из оставшихся и меняем местами со вторым. Повторяем это до тех пор, пока не останется последний элемент в списке.



Сортировка выбором (Selection Sort)

```
Исходный массив: [5, 3, 8, 1]
Шаг 1: ищем минимум среди всех → 1, меняем с 5 → [1, 3, 8, 5]
Шаг 2: ищем минимум среди [3, 8, 5] → 3 уже на месте → [1, 3, 8, 5]
Шаг 3: ищем минимум среди [8, 5] → 5, меняем с 8 → [1, 3, 5, 8]
```

Сортировка выбором (Selection Sort)

Характеристики:

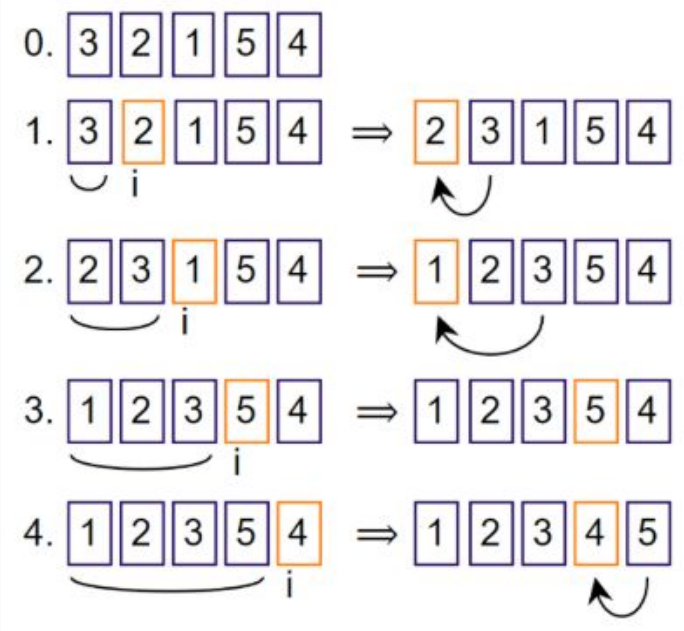
- Сложность: $O(n^2)$ — всегда (даже для отсортированного массива)
- Память: $O(1)$
- Стабильность: Нет (обмен может нарушить порядок равных элементов)

Сортировка вставками (Insertion Sort)

Сортировка вставками быстрее и проще двух предыдущих. Именно так большинство людей тасует карты любой игре. На каждой итерации программа берет один из элементов и подыскивает для него место в уже отсортированном списке. Так происходит до тех пор, пока не останется ни одного неиспользованного элемента.

Предполагается, что первый элемент списка отсортирован. Переходим к следующему элементу, обозначим его x. Если x больше первого, оставляем его на своём месте. Если он меньше, копируем его на вторую позицию, а x устанавливаем как первый элемент.

Переходя к другим элементам несортированного сегмента, перемещаем более крупные элементы в отсортированном сегменте вверх по списку, пока не встретим элемент меньше x или не дойдём до конца списка. В первом случае x помещается на правильную позицию.



Сортировка вставками (Insertion Sort)

```
Исходный массив: [5, 3, 8, 1, 4]
Шаг 1: [5] отсортирован, берём 3 → вставляем перед 5 → [3, 5, 8, 1, 4]
Шаг 2: [3, 5] отсортированы, 8 на месте → [3, 5, 8, 1, 4]
Шаг 3: 1 меньше всех → сдвигаем 8,5,3 вправо → [1, 3, 5, 8, 4]
Шаг 4: 4 вставляем между 3 и 5 → [1, 3, 4, 5, 8]
```

Сортировка вставками (Insertion Sort)

Характеристики:

- Сложность: $O(n^2)$ — худший, $O(n)$ — лучший (почти отсортированный массив)
- Память: $O(1)$
- Стабильность: Да
- Отлично работает на маленьких или почти отсортированных массивах

Сортировка слиянием (Merge Sort)

Сортировка слиянием — элегантный пример использования подхода «Разделяй и властвуй». Он состоит из двух этапов:

1. Несортированный список последовательно делится на N списков, где каждый включает один «несортированный» элемент, а N — это число элементов в оригинальном массиве.
2. Списки последовательно сливаются группами по два, создавая новые отсортированные списки до тех пор, пока не появится один финальный отсортированный список.

В отличие от предыдущих алгоритмов, рассматриваемый алгоритм возвращает новый список, а не сортирует существующий. Поэтому такая сортировка требует больше памяти для создания нового списка того же размера, что и входной список.

В среднем время сортировки слиянием составляет O(n log n)

Сортировка слиянием (Merge Sort)

Список рекурсивно разделяется пополам, пока в итоге не получатся списки размером в один элемент. Массив из одного элемента считается упорядоченным. Соседние элементы сравниваются и соединяются вместе. Это происходит до тех пор, пока не получится полный отсортированный список.

Сортировка осуществляется путём сравнения наименьших элементов каждого подмассива. Первые элементы каждого подмассива сравниваются первыми. Наименьший элемент перемещается в результирующий массив. Счётчики результирующего массива и подмассива, откуда был взят элемент, увеличиваются на 1.

```
[5, 3, 8, 1, 4, 2]
          ↓
[5, 3, 8] и [1, 4, 2]
   ↓            ↓
[5] [3,8]    [1] [4,2]
    ↓           ↓
   [3,8]       [2,4]
     ↓           ↓
   [3,5,8] и [1,2,4]
          ↓
     [1,2,3,4,5,8]
```

Сортировка слиянием (Merge Sort)

Характеристики:

- Сложность: O(n log n) — всегда
- Память: O(n) — требует дополнительный массив
- Стабильность: Да
- Минус: требует дополнительной памяти

Быстрая сортировка (Quick Sort)

Этот алгоритм также относится к алгоритмам «разделяй и властвуй». Его используют чаще других алгоритмов. При правильной конфигурации он чрезвычайно эффективен и не требует дополнительной памяти, в отличие от сортировки слиянием. Массив разделяется на две части по разные стороны от опорного элемента. В процессе сортировки элементы меньше опорного помещаются перед ним, а равные или большие — позади.

Алгоритм состоит из трех этапов:

1. Выбирается один опорный элемент.
2. Все элементы меньше опорного перемешаются слева от него, остальные — направо. Это называется операцией разбиения.
3. Рекурсивно повторяются 2 предыдущих шага к каждому новому списку, где новые опорные элементы будут меньше и больше оригинального соответственно.

В среднем время выполнения быстрой сортировки составляет $O(n \log n)$.
Обратите внимание, что алгоритм быстрой сортировки будет работать медленно, если опорный элемент равен наименьшему или наибольшему элементам списка. При таких условиях, в отличие от сортировок кучей и слиянием, обе из которых имеют в худшем случае время сортировки $O(n \log n)$, быстрая сортировка в худшем случае будет выполняться $O(n^2)$.

Быстрая сортировка (Quick Sort)

```
[5, 3, 8, 1, 4, 2]  выбираем опорный 4
            ↓
Меньше 4: [3,1,2]  Больше 4: [5,8]
    ↓                       ↓
[1,2,3] + [4] + [5,8] → [1,2,3,4,5,8]
```

5	6	2	-2	6	10	12	15

Быстрая сортировка (Quick Sort)

Характеристики:

- Сложность: $O(n \log n)$ — средний, $O(n^2)$ — худший (неудачный выбор опорного)
- Память: $O(\log n)$ — для рекурсии
- Стабильность: Нет (в типичной реализации)
- Самый быстрый на практике при хорошем выборе опорного

Пирамидальная сортировка (Heap Sort)

Также известна как сортировка кучей. Этот популярный алгоритм, как и сортировки вставками или выборкой, сегментирует список на две части: отсортированную и неотсортированную. Алгоритм преобразует второй сегмент списка в структуру данных «куча» (heap), чтобы можно было эффективно определить самый большой элемент.

Сначала преобразуем список в **Max Heap** — бинарное дерево, где самый большой элемент является вершиной дерева. Затем помещаем этот элемент в конец списка. После перестраиваем **Max Heap** и снова помещаем новый наибольший элемент уже перед последним элементом в списке.

Этот процесс построения кучи повторяется, пока все вершины дерева не будут удалены.

В среднем время сортировки кучей составляет $O(n \log n)$, что уже значительно быстрее предыдущих алгоритмов

Пирамидальная сортировка (Heap Sort)

```
[5, 3, 8, 1, 4, 2]
        ↓
Строим кучу: [8, 4, 5, 1, 3, 2]
        ↓
Меняем 8 с последним: [2, 4, 5, 1, 3, 8]
        ↓
Восстанавливаем кучу для [2,4,5,1,3] → [5,4,2,1,3]
        ↓
Меняем 5 с предпоследним: [3,4,2,1,5,8]
... продолжается до полной сортировки
```

Пирамидальная сортировка (Heap Sort)

Характеристики:

- Сложность: O(n log n) — всегда
- Память: O(1)
- Стабильность: Нет
- Преимущество: гарантированное время и экономия памяти

Сравнение алгоритмов

Алгоритм	Лучший случай	Средний	Худщий	Память	Стабильность
Пузырьковая	O(n)	O(n^2)	O(n^2)	O(1)	Да
Выбором	O(n^2)	O(n^2)	O(n^2)	O(1)	Нет
Вставками	O(n)	O(n^2)	O(n^2)	O(1)	Да
Слиянием	O(n log n)	O(n log n)	O(n log n)	O(n)	Да
Быстрая	O(n log n)	O(n log n)	O(n^2)	O(log n)	Нет
Пирамидальная	O(n log n)	O(n log n)	O(n log n)	O(1)	Нет

Сравнение скоростей (экспериментальные данные)

Размер	Пузырьк.	Выбором	Вставками	Пирамид.	Слиянием	Быстрая
1000	0.0315	0.0315	0.0156	0.0004	0.0003	0.0002
2000	0.1250	0.1094	0.0625	0.0009	0.0007	0.0005
5000	0.7813	0.7031	0.3906	0.0025	0.0019	0.0012
10000	3.1250	2.8125	1.5625	0.0053	0.0042	0.0027

Время в секундах, зависит от реализации и оборудования

Вывод

- Пузырьковая, выбором, вставками — только для обучения
- Быстрая — лидер на практике
- Пирамидальная — гарантированное время
- Слияние — стабильность ценой памяти

Встроенные функции сортировки в Python

Python предоставляет два основных способа сортировки:

- Метод sort() — сортирует список на месте (изменяет оригинал):

```
numbers = [5, 2, 8, 1, 9]
numbers.sort()
print(numbers)  # [1, 2, 5, 8, 9]

# Сортировка по убыванию
numbers.sort(reverse=True)
print(numbers)  # [9, 8, 5, 2, 1]
```

Встроенные функции сортировки в Python

- Функция sorted() — возвращает новый отсортированный список:

```
original = [5, 2, 8, 1, 9]
sorted_numbers = sorted(original)
print(original)       # [5, 2, 8, 1, 9] — без изменений
print(sorted_numbers) # [1, 2, 5, 8, 9]

# Сортировка по убыванию
sorted_desc = sorted(original, reverse=True)
Сортировка сложных структур:

# Сортировка словарей по ключу
students = [
    {'name': 'Анна', 'grade': 85},
    {'name': 'Иван', 'grade': 92},
    {'name': 'Мария', 'grade': 78}
]
by_grade = sorted(students, key=lambda x: x['grade'])
print(by_grade)
# [{'name': 'Мария', 'grade': 78}, {'name': 'Анна', 'grade': 85}, {'name': 'Иван', 'grade': 92}]

# Сортировка строк по длине
words = ["python", "java", "c", "javascript"]
by_length = sorted(words, key=len)
print(by_length)  # ['c', 'java', 'python', 'javascript']
```

Рекомендации по выбору алгоритма

Ситуация	Рекомендуемый алгоритм	Причина
Небольшой массив (до 50)	Вставками	Простота, эффективность для малых данных
Почти отсортированный массив	Вставками	O(n) в лучшем случае
Огромные данные, важна гарантия	Пирамидальная или слиянием	O(n log n) всегда
Важна стабильность	Слиянием или вставками	Сохраняют порядок равных
Максимальная скорость на практике	Быстрая	Лучшие показатели в среднем
Ограничена память	Пирамидальная	O(1) дополнительной памяти
Учебные цели	Все простые	Понимание принципов

Рекомендации по выбору алгоритма

Золотое правило: В реальных проектах используйте встроенные функции sort() и sorted() — они реализованы на высокоэффективном алгоритме TimSort (гибрид сортировки вставками и слиянием).

Заключение

- Сортировка — фундаментальная задача в программировании
- Разные алгоритмы имеют разные характеристики
- Для учебных целей важно понимать принципы работы
- В реальных проектах используйте встроенные функции
- Выбор алгоритма зависит от конкретной задачи